Physics-Informed Neural Networks

Literature Review for Track Extrapolation

G. Scriven

Maastricht University · UHasselt · LHCb

March 2026

Outline

1. Motivation
2. Original PINN Papers
3. Paper 1: FD-PINN
4. Paper 2: Data-Free PINNs
5. Paper 3: DeepONet Domain Decomposition
6. Paper 4: RK-PINN
7. Additional Literature
8. Synthesis & Roadmap
9. Conclusions

The Track Extrapolation Problem

Goal: Replace C++ RK4 track extrapolator with a neural network.

Lorentz force ODEs:

$$\frac{dt_x}{dz} = \kappa N\left[t_x t_y B_x - (1+t_x^2)B_y + t_y B_z\right]$$
$$\frac{dt_y}{dz} = \kappa N\left[(1+t_y^2)B_x - t_x t_y B_y - t_x B_z\right]$$

where $\kappa = (q/p) \cdot c_{\text{light}}$, $N = \sqrt{1 + t_x^2 + t_y^2}$.

Learn the mapping:

$$f_\theta : (x_0, y_0, t_{x,0}, t_{y,0}, q/p, \Delta z) \to (x_f, y_f, t_{x,f}, t_{y,f})$$

V1 Key Results

Model	μ**s**	**Error**
MLP tiny	1.12	53 μm
MLP medium	2.17	37 μm
PINN	2.46	593 mm
RK-PINN	5.59	2 km
C++ RK4	2.50	0 (ref.)

PINNs failed!

Errors $10^4\times$ worse than MLPs.
Root cause: IC violation bug.

Why PINNs Failed in V1

The bug: `forward()` hardcoded $z_{\text{frac}} = 1.0$

- Network never saw intermediate trajectory points
- Physics loss at collocation points evaluated *untrained* regime
- Output independent of $z \Rightarrow$ IC violated

Additional issues:

- Gaussian field approximation $\neq$ real field map
- Only weak t_y conservation constraint
- 50M samples made data loss dominant

Proposed Fix: PINN_v3

Residual architecture guarantees IC:

$$\boldsymbol{y}(z) = \boldsymbol{y}_0 + z_{\text{frac}} \cdot \text{MLP}(\boldsymbol{y}_0, q/p, \Delta z)$$

- At $z_{\text{frac}} = 0$: $\boldsymbol{y}(0) = \boldsymbol{y}_0$ ✓
- Smooth interpolation guaranteed
- Not yet trained in V1

Question: What does the literature say about these issues?

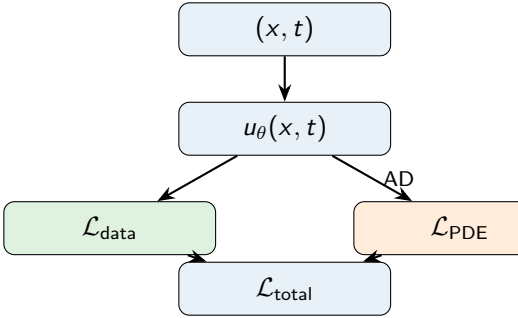
The PINN Framework

Raissi, Perdikaris & Karniadakis (2019) — *J. Comput. Phys.*

- Neural network $u_\theta(x, t)$ approximates PDE solution
- Physics residual added to loss function:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{data}}}_{\text{boundary/IC}} + \lambda \underbrace{\mathcal{L}_{\text{PDE}}}_{\text{residual at collocation pts}}$$

- Derivatives computed via automatic differentiation
- Demonstrated on Burgers', Schrödinger, Navier–Stokes
- **Foundational paper** — 10,000+ citations



Paper 1: Finite Difference PINNs

"Physics-informed deep learning based on the finite difference method for efficient and accurate numerical solution of PDEs"

Core idea: Replace automatic differentiation with finite differences.

$$\left.\frac{\partial \hat{u}}{\partial x}\right|_{x_i} \approx \frac{\hat{u}(x_i+\delta) - \hat{u}(x_i-\delta)}{2\delta}$$

Advantages:

- No computational graph storage $\Rightarrow$ lower memory
- Faster training (FD $\approx$ forward passes only)
- Works with **non-differentiable** components
- Tested on Burgers', convection–diffusion, Navier–Stokes

Direct relevance to V1

V1 was **forced** to use a Gaussian field model because AD requires differentiability:

$$B_y(z) = B_0\, e^{-(z-z_c)^2/2\sigma_z^2}$$

With FD, the tabulated field map (`twodip.rtf`) can be used **directly**.

Impact

Eliminates the field model mismatch — the dominant V1 physics loss issue.

Paper 2: Learning Without Data

Misyris, Stiasny & Chatzivasileiadis — IEEE SmartGridComm (2021)

Core idea: Train PINNs with *zero* training data.

$$\mathcal{L} = \lambda_{\text{PDE}}\,\mathcal{L}_{\text{PDE}} + \lambda_{\text{IC}}\,\mathcal{L}_{\text{IC}} + \lambda_{\text{BC}}\,\mathcal{L}_{\text{BC}}$$

Key results:

- Solved DAEs for power grid simulation
- Orders-of-magnitude speedup over numerical solvers
- Single network solves families of ICs
- No data generation infrastructure needed

Comparison with V1

	V1	**Paper 2**
Data	50M	**0**
Input dim	6D	1–3D
Loss	data+PDE	PDE only

Implication

Pure data-free unlikely for 6D LHCb problem.

Hybrid: 1–5M samples + physics loss for regularisation.

Paper 3: Long-Time Integration with DeepONets

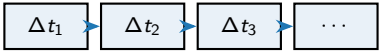
"Long-time integration of parametric evolution equations with physics-informed DeepONets"

Problem: PINNs fail for long integration horizons (spectral bias).

Solution: DeepONet + temporal domain decomposition.

- DeepONet learns the *solution operator* $\mathcal{G} : u_0 \mapsto u(t)$
- Branch net encodes input function; trunk net encodes output locations
- Time domain split into short windows; predictions chained
- Handles parametric families of ICs

chain predictions



Direct relevance to V1

V1 fixed $\Delta z = 8000\,\text{mm}$. Could not generalise to variable step sizes.

Chaining short steps solves this naturally.

Paper 4: Runge–Kutta Structured PINNs

"Parameter estimation and modeling of nonlinear dynamical systems based on RK-PINN"

Core idea: Embed RK integration structure into the network.

$$\hat{\boldsymbol{s}}_{\text{out}} = \boldsymbol{s}_{\text{in}} + \sum_{k=1}^{4} w_k \, \Delta \boldsymbol{s}_k$$

- Each stage predicts derivative at intermediate point
- Weights initialised to RK4 Butcher tableau
- Strong inductive bias ⇒ respects ODE structure
- Dual: forward simulation + inverse parameter estimation

V1 RK-PINN vs. Literature

	V1	**Paper 4**
IC guarantee	**No**	**Yes**
Physics loss	t_y only	Full PDE
Accuracy	2 km	Accurate
Inverse	No	Yes

Inverse capability

Could estimate q/p from track hits ⇒ unify extrapolation & fitting.

PINN Failure Modes

Krishnapriyan, Gholami, Zhe, Kirby & Mahoney — *NeurIPS* (2021)
"Characterizing possible failure modes in physics-informed neural networks"

- PINNs fail on even *slightly* complex problems
- Soft PDE regularisation makes loss landscape ill-conditioned
- Failure is **not** due to limited expressivity — it's **optimisation**
- Tested: convection, reaction, diffusion operators

Two proposed solutions:

1. **Curriculum regularisation:** start simple, increase complexity
2. **Sequence-to-sequence:** predict windows, not full space-time

Explains V1 failure

V1's coupled 4-ODE system with non-uniform field is exactly the "complex enough to fail" regime.

Curriculum warmup was used in V1 (10 epochs) — but insufficient without the IC fix.

Remedy

Sequence-to-sequence $\equiv$ domain decomposition (Paper 3).

Neural Operators: DeepONet & FNO

DeepONet (Lu, Jin & Karniadakis, 2019)

- Learns **operators** between function spaces
- Branch net: encodes input function at sensors
- Trunk net: encodes output locations
- High-order error convergence (up to 4th order)
- Published in *Nature Machine Intelligence* (2021)

Relevance: Could learn $\mathcal{G} : \boldsymbol{s}_0 \mapsto \boldsymbol{s}(z)$ for *all* initial states & momenta at once.

FNO (Li et al., 2020)

- Parameterises integral kernel in Fourier space
- Up to 1000× faster than PDE solvers
- Zero-shot super-resolution
- Tested: Burgers', Darcy flow, Navier–Stokes
- Published at *ICLR* (2021)

Relevance: Alternative operator architecture. Works best for PDE fields on grids — less natural for ODE trajectories.

Lu et al. (2022, *CMAME*): DeepONet more robust to noise than FNO on 16 benchmarks.

V1 Failures $\leftrightarrow$ Literature Solutions

V1 Failure Mode	**Paper**	**Proposed Solution**
IC violation (PINN ignores z)	2, 4, Failure modes	Residual architecture (PINN_v3)
Field model mismatch	1	FD-based physics loss
Fixed Δz only	3	Temporal domain decomposition
Weak physics constraint	4	Full PDE residual at each stage
50M samples required	2	Hybrid: few-M data + physics
No inverse capability	4	Dual forward/inverse RK-PINN

Priority Roadmap for V2+

P1 Fix & retrain PINN_v3 + RK-PINN_v3

- Residual architecture: $\boldsymbol{y}(z) = \boldsymbol{y}_0 + z_{\text{frac}} \cdot \text{MLP}(\cdots)$
- **Prerequisite** for all other improvements

P2 Adopt FD-based physics loss (Paper 1)

- Use production field map directly — no more Gaussian approximation
- Low implementation effort; immediate impact

P3 Short-step chaining (Paper 3)

- Train accurate $\Delta z = 500$–$1000\,\text{mm}$ model
- Chain N steps for longer extrapolations
- Compatible with Kalman filter's sequential structure

P4 Inverse problem unification (Paper 4) — exploratory

- Estimate q/p from track hits; refine field map; detector alignment
- Unify extrapolation + fitting in a single differentiable model

Conclusions

1. **V1 MLPs work:** 2.9× speedup, sub-100 μm accuracy.
2. **V1 PINNs failed due to a fixable bug**, not a fundamental limitation.
3. **The literature provides solutions for every V1 failure mode:**
   - FD-PINNs → use real field map (no Gaussian mismatch)
   - Residual architectures → guarantee IC satisfaction
   - Domain decomposition → variable step sizes
   - Curriculum training → stabilise complex ODE landscapes
   - RK-structure → encode numerical integration as inductive bias
4. **Most promising path:** PINN_v3 + FD physics loss + short-step chaining.
5. **Operator learning** (DeepONet/FNO) is a longer-term opportunity for parametric generalisation.

References I

- M. Raissi, P. Perdikaris, G.E. Karniadakis, "Physics-informed neural networks," *J. Comput. Phys.* **378**, 686 (2019).
- "Physics-informed deep learning based on the finite difference method for efficient and accurate numerical solution of PDEs."
- G.S. Misyris, A. Stiasny, S. Chatzivasileiadis, "Learning without Data: PINNs for Fast Time-Domain Simulation," *IEEE SmartGridComm* (2021).
- "Long-time integration of parametric evolution equations with physics-informed DeepONets."
- "Parameter estimation and modeling of nonlinear dynamical systems based on RK-PINN."
- A.S. Krishnapriyan et al., "Characterizing possible failure modes in PINNs," *NeurIPS* (2021). arXiv:2109.01050.
- L. Lu, P. Jin, G.E. Karniadakis, "DeepONet: Learning nonlinear operators," *Nature Mach. Intell.* **3**, 218 (2021).

References II

Z. Li et al., "Fourier Neural Operator for Parametric PDEs," *ICLR* (2021). arXiv:2010.08895.

R.T.Q. Chen et al., "Neural Ordinary Differential Equations," *NeurIPS* (2018). arXiv:1806.07366.

Thank you

Questions?

George.Scriven@uhasselt.be
George.William.Scriven@cern.ch